Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования «Казанский (Приволжский) федеральный университет»

ИНСТИТУТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ
И ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

КАФЕДРА ПРИКЛАДНОЙ МАТЕМАТИКИ
И ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

Направление: 01.03.04 — Прикладная математика

Профиль: Прикладная математика

ОТЧЁТ ПО ПРОЕКТУ

Многослойный персептрон

Студент 3 курса
группы 09-321
«_____» _____ 2026 г. Е.С. Уфаев

Преподаватель
«_____» _____ 2026 г. _____

Казань — 2026

Содержание

ВВЕДЕНИЕ 1
1 Механизмы обучения 2
1.1 Прямое распространение 2
1.2 Функция потерь 2
1.3 Обратное распространение через автоматическое дифференцирование 2
1.4 Градиентный спуск 3
1.5 Подготовка данных и организация обучения 4
2 Программная реализация 5
2.1 Структура библиотеки 5
2.2 Используемые библиотеки 5
2.3 Интерфейс 5
3 Экспериментальный выбор архитектуры 6
3.1 Методика 6
3.2 Результаты сравнения 6
3.3 Выводы 7
4 Результаты 8
4.1 Процесс обучения 8
4.2 Итоговое качество 9
ЗАКЛЮЧЕНИЕ 10
ПРИЛОЖЕНИЕ А 11

ВВЕДЕНИЕ

Искусственные нейронные сети являются основным инструментом решения задач классификации, для которых не удаётся явно сформулировать решающее правило. Простейшей и исторически первой архитектурой такого рода является многослойный персептрон — последовательность полносвязных слоёв, разделённых нелинейными функциями активации. Несмотря на простоту устройства, именно на нём отрабатываются все базовые механизмы обучения: прямое распространение сигнала, вычисление функции потерь, обратное распространение ошибки и градиентный спуск.

Практическая часть работы выполнена на языке C++ и включает библиотеку с реализацией перечисленных механизмов и серию вычислительных экспериментов по выбору архитектуры сети.

Постановка задачи. Разработать многослойный персептрон с настраиваемой архитектурой, обучаемый методом обратного распространения ошибки и градиентного спуска, и применить его к задаче бинарной классификации диагноза по тридцати признакам из набора.

1 Механизмы обучения

1.1 Прямое распространение

Сеть представляет собой цепочку полносвязных слоёв, разделённых нелинейностями. Для слоя с входом X размера $[N \times d_{\text{in}}]$, где N — число примеров в обрабатываемой порции данных, выход вычисляется как

$$Z = XW + b, \qquad A = f(Z), \tag{1}$$

где W — матрица весов размера $[d_{\text{in}} \times d_{\text{out}}]$, b — вектор смещений размера $[1 \times d_{\text{out}}]$, прибавляемый ко всем N строкам, а f — функция активации, применяемая поэлементно.

На выходном слое применяется функция `softmax`, преобразующая вектор произвольных вещественных чисел в распределение вероятностей по классам:

$$p_i = \frac{\exp\bigl(z_i - \max_k z_k\bigr)}{\sum_j \exp\bigl(z_j - \max_k z_k\bigr)}. \tag{2}$$

Вычитание максимума по строке не меняет результата, поскольку числитель и знаменатель делятся на одну и ту же константу, однако предотвращает переполнение при вычислении экспоненты от больших аргументов.

1.2 Функция потерь

В качестве функции потерь используется бинарная кросс-энтропия, вычисляемая по вероятности положительного класса:

$$L = -\frac{1}{N} \sum_{n=1}^{N} \Bigl[\, y_n \log p_n + (1 - y_n) \log(1 - p_n) \,\Bigr]. \tag{3}$$

Вероятности предварительно ограничиваются отрезком $[\varepsilon,\ 1 - \varepsilon]$ при $\varepsilon = 10^{-12}$: в противном случае уверенный и при этом неверный прогноз даёт $\log 0 = -\infty$, что приводит к появлению неопределённых значений во всех градиентах. Помимо бинарной кросс-энтропии реализованы категориальная кросс-энтропия и среднеквадратичная ошибка.

Деление на число примеров N выполняется внутри функции потерь. Благодаря этому величина шага градиентного спуска сохраняет смысл при любом размере обрабатываемой порции данных.

1.3 Обратное распространение через автоматическое дифференцирование

Формулы обратного прохода для слоя в программе не записаны. Вместо них заданы правила дифференцирования отдельных элементарных операций, а производные по всем параметрам выводятся программно.

Каждая операция над тензором вычисляет не только результат, но и создаёт узел графа вычислений, хранящий ссылки на узлы-аргументы и правило передачи градиента к ним. Граф, таким образом, является побочным продуктом прямого прохода: отдельного этапа построения графа не выполняется. Рёбра графа направлены от результата к аргументам, владение узлами организовано в том же направлении, поэтому граф существует до тех пор, пока существует ссылка на его корень, и освобождается целиком при её удалении.

Обратный проход состоит из трёх этапов. Сначала граф обходится в обратном топологическом порядке; такой порядок гарантирует, что к моменту применения правила очередного узла в его градиент уже просуммированы вклады всех узлов, которые этот узел использовали. Затем в корень графа записывается начальное значение $\partial L/\partial L = 1$. После этого каждое правило получает накопленный градиент и прибавляет свой вклад к градиентам аргументов.

Накопление градиента выполняется суммированием. Поэтому узел, использованный в нескольких ветвях сети, автоматически получает сумму вкладов, как того требует правило дифференцирования сложной функции. Из этого же свойства следует, что модель не обязана быть линейной цепочкой: на нём основана работа блока с обходным соединением. Прямым следствием такого устройства является то, что базовый интерфейс обучаемого блока содержит только прямой проход — метод обратного прохода отсутствует у всех слоёв.

Для слоя, задаваемого выражением $Z = XW + b$, описанный механизм приводит к следующим выражениям:

$$\frac{\partial L}{\partial W} = X^{\mathsf{T}}\,\frac{\partial L}{\partial Z}, \qquad \frac{\partial L}{\partial b} = \sum_{n=1}^{N} \frac{\partial L}{\partial Z_{n,:}}, \qquad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z}\,W^{\mathsf{T}}. \tag{4}$$

Градиент по вектору смещений представляет собой сумму по строкам, поскольку один и тот же вектор b участвует в вычислении всех N строк результата.

Для поэлементных функций активации градиент умножается на производную соответствующей функции:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big), \qquad \tanh'(z) = 1 - \tanh^2(z), \qquad \mathrm{ReLU}'(z) = [\,z > 0\,]. \tag{5}$$

1.4 Градиентный спуск

После обработки каждой порции данных параметры смещаются в направлении, противоположном градиенту:

$$W \leftarrow W - \eta\,\frac{\partial L}{\partial W}, \tag{6}$$

где η — скорость обучения. Реализован также вариант с накоплением импульса, сглаживающий траекторию спуска:

$$v \leftarrow \mu\,v - \eta\,\frac{\partial L}{\partial W}, \qquad W \leftarrow W + v. \tag{7}$$

1.5 Подготовка данных и организация обучения

Набор данных содержит 569 примеров, по 30 вещественных признаков в каждом, при соотношении классов 357 к 212 (62,74 % и 37,26 %). Признаки измеряются в несопоставимых единицах: среднее квадратичное отклонение по столбцам различается более чем на пять порядков. Поскольку градиент по весу пропорционален соответствующему входу, без приведения признаков к общему масштабу подобрать единую скорость обучения для всей сети невозможно. Поэтому признаки стандартизуются:

$$x' = \frac{x - \mu}{\sigma}, \tag{8}$$

причём параметры μ и σ вычисляются только по обучающей выборке и сохраняются вместе с моделью.

Выборка делится в соотношении 80 к 20 со стратификацией по классам, что даёт 456 примеров для обучения и 113 для проверки.

Одна эпоха представляет собой полный проход по обучающей выборке порциями по восемь примеров. Перед каждой эпохой порядок примеров заново перемешивается, иначе последовательность шагов спуска повторялась бы от эпохи к эпохе. Шаг обучения состоит из обнуления градиентов, прямого прохода, вычисления функции потерь, обратного прохода и шага оптимизатора. Обнуление необходимо именно потому, что градиенты накапливаются суммированием. По завершении каждой эпохи вычисляются функция потерь и доля правильных ответов на обеих выборках; эти вычисления выполняются в режиме, при котором граф не строится. Дополнительно реализована ранняя остановка с возвратом к параметрам лучшей эпохи.

Единое seed генератора случайных чисел используется при инициализации весов, разделении выборки и перемешивании перед каждой эпохой, что обеспечивает воспроизводимость результата.

2 Программная реализация

2.1 Структура библиотеки

Библиотека реализована на языке C++ стандарта C++23 и состоит из 25 заголовочных файлов, сгруппированных по подсистемам:

1) `core` — матричный тип и генератор случайных чисел с явным сидом;

2) `autograd` — механизм автоматического дифференцирования: узел графа, обратный проход и четырнадцать операций с правилами их дифференцирования;

3) `data` — загрузка выборки, стандартизация, стратифицированное разделение, унитарное кодирование меток и статистический анализ признаков;

4) `nn` — обучаемые блоки: полносвязный слой, функции активации, функции потерь, схемы инициализации весов и модель целиком;

5) `optim` — интерфейс оптимизатора и градиентный спуск с импульсом;

6) `train` — цикл обучения, история метрик по эпохам и метрики качества: доля правильных ответов, матрица ошибок, точность;

7) `io` — сохранение и загрузка модели в текстовом формате, включающем архитектуру сети, веса и параметры стандартизации;

8) `viz` — построение кривых обучения.

Разбор аргументов командной строки вынесен в отдельный файл `apps/main.cpp`. Листинги ключевых фрагментов приведены в приложении А.

2.2 Используемые библиотеки

Для вспомогательных задач, не относящихся к предмету работы, применяются готовые библиотеки: Eigen 3.4.0 для операций линейной алгебры, Matplot++ 1.2.2 для построения графиков, fmt 11.0.2 для форматирования вывода, rapidcsv 9.07 для чтения и записи файлов формата CSV, CLI11 2.7.2 для разбора командной строки и Catch2 3.9.1 для модульного тестирования.

2.3 Интерфейс

Программа оформлена в виде единого исполняемого файла с четырьмя подкомандами: `describe` выполняет статистический анализ набора данных, `split` разделяет его на обучающую и проверочную выборки, `train` обучает сеть и сохраняет модель, `predict` загружает сохранённую модель, выполняет прогноз и оценивает его по бинарной кросс-энтропии.

Архитектура и параметры обучения задаются аргументами командной строки: число и ширина скрытых слоёв, функция активации, схема инициализации, функция потерь, число эпох, размер порции данных, скорость обучения, коэффициент импульса, терпение ранней остановки и зерно генератора случайных чисел.

3 Экспериментальный выбор архитектуры

3.1 Методика

Задание допускает произвольную конфигурацию сети при условии наличия не менее двух скрытых слоёв, поэтому архитектура выбиралась экспериментально: сравнивались шесть конфигураций, различающихся глубиной, шириной и функцией активации скрытых слоёв.

Условия сравнения были уравнены: разбиение выборки фиксировано — одно и то же стратифицированное разделение 456 к 113 при сиде 42, параметры стандартизации вычислены по обучающей части. Все конфигурации обучались 84 эпохи порциями по 8 примеров при $\eta = 0{,}0314$, бинарной кросс-энтропии, инициализации He и нулевых смещениях.

Результат одного запуска зависит от сида, определяющего начальные веса и порядок перемешивания, причём разброс по сиду сопоставим с различиями между конфигурациями. Поэтому каждая конфигурация обучалась пять раз с зёрнами 1, 7, 42, 123 и 2026, в сравнении участвуют средние значения, а дополнительно вычислялось среднее квадратичное отклонение функции потерь на проверке — оно показывает, какие различия в таблице значимы. Сравнение оформлено как автоматический тест.

3.2 Результаты сравнения

Результаты приведены в таблице 1. Величины L и L_{val} обозначают функцию потерь на обучающей и проверочной выборках соответственно, s — среднее квадратичное отклонение L_{val} по пяти запускам, A_{val} — доля правильных ответов на проверочной выборке.

Таблица 1 — Сравнение архитектур, средние значения по пяти запускам

Архитектура	Параметров	L	L_{val}	s	A_{val}
24–24, сигмоида	1394	0,0519	0,0792	0,0018	0,9664
24–24–24, сигмоида	1994	0,0515	0,0821	0,0046	0,9646
48–48, сигмоида	3938	0,0530	0,0768	0,0037	0,9646
8–8, сигмоида	338	0,0522	0,0788	0,0075	0,9664
24–24, ReLU	1394	0,0034	0,0674	0,0293	0,9752
24–24, тангенс	1394	0,0075	**0,0265**	0,0097	**0,9858**

3.3 Выводы

Ёмкость сети не является ограничивающим фактором. Четыре конфигурации с сигмоидой различаются по числу параметров в двенадцать раз — от 338 до 3938, — однако функция потерь на проверке меняется лишь в пределах 0,0768–0,0821, что сопоставимо со средним квадратичным отклонением по сиду (до 0,0075) и потому незначимо. Решающей является функция потерь на *обучающей* выборке: она составляет около 0,052 у всех четырёх конфигураций независимо от размера. Будь сеть ограничена ёмкостью, двенадцатикратный рост числа параметров позволил бы лучше приблизить обучающую выборку. Этого не происходит, значит ограничение лежит не в выразительной способности модели, а в сходимости: при заданных скорости обучения и числе эпох обучение не успевает завершиться.

Добавление слоя ухудшает результат: переход от двух скрытых слоёв к трём при той же ширине увеличивает число параметров с 1394 до 1994, но функция потерь на проверке растёт с 0,0792 до 0,0821, а разброс по сиду — с 0,0018 до 0,0046. Причина та же: производная сигмоиды не превосходит 0,25, поэтому каждый дополнительный слой уменьшает проходящий градиент не менее чем вчетверо. Третий слой замедляет сходимость, не давая взамен выигрыша в качестве.

Определяющим параметром оказалась функция активации: замена сигмоиды на гиперболический тангенс при неизменной архитектуре снижает функцию потерь на проверке втрое — с 0,0792 до 0,0265, — повышает долю правильных ответов с 0,9664 до 0,9858, а на обучающей выборке даёт падение с 0,0519 до 0,0075, то есть сеть действительно начинает сходиться. Объяснение следует из (5): максимум производной тангенса равен единице против 0,25 у сигмоиды, что даёт четырёхкратный выигрыш в величине градиента на каждом слое. Кроме того, тангенс симметричен относительно нуля, тогда как сигмоида выдаёт только положительные значения, из-за чего компоненты градиента по весам одного нейрона сонаправлены и спуск идёт зигзагом.

ReLU сходится быстрее, но обобщает хуже: она даёт наименьшую из всех функцию потерь на обучающей выборке (0,0034), но на проверочной — 0,0674: разрыв между выборками 0,064 против 0,019 у тангенса, то есть переобучение, а разброс по сиду ($s = 0{,}0293$) втрое больше. Причина — обнуление производной на отрицательной полуоси: часть нейронов при неудачной инициализации перестаёт получать градиент, и итог существенно зависит от начальных весов.

Принята конфигурация из двух скрытых слоёв по 24 нейрона с гиперболическим тангенсом: лучшие значения обеих метрик, наименьший разрыв между выборками среди сошедшихся конфигураций, не менее двух скрытых слоёв. Она установлена в программе как используемая по умолчанию.

4 Результаты

4.1 Процесс обучения

Итоговая модель имеет структуру $30 \to 24 \to 24 \to 2$ с гиперболическим тангенсом в скрытых слоях и функцией `softmax` на выходе, что составляет 1394 обучаемых параметра. Обучение выполнялось в течение 84 эпох порциями по восемь примеров при $\eta = 0{,}0314$ и зерне 42.

Кривые обучения приведены на рисунке 1.

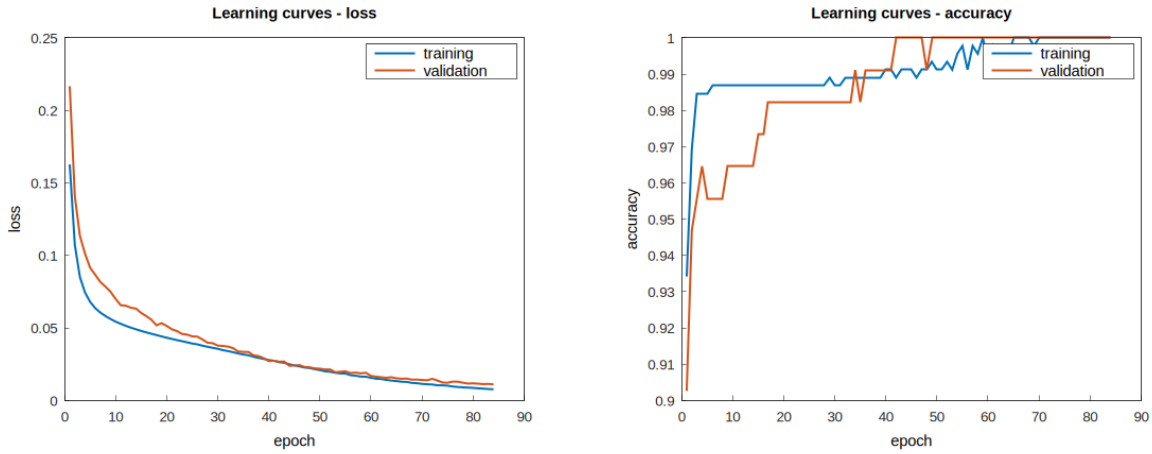


Рисунок 1 — Функция потерь и доля правильных ответов по эпохам

Обе кривые функции потерь монотонно убывают и сближаются по ходу обучения. Динамика разрыва между ними приведена в таблице 2.

Таблица 2 — Функция потерь по эпохам и разрыв между выборками

Эпоха	L	L_{val}	Разрыв
1	0,1628	0,2167	0,0539
12	0,0515	0,0654	0,0139
24	0,0401	0,0454	0,0053
36	0,0312	0,0335	0,0023
48	0,0224	0,0229	0,0005
60	0,0157	0,0170	0,0013
72	0,0110	0,0150	0,0039
84	0,0077	0,0111	0,0034

Разрыв сокращается с 0,0539 на первой эпохе до 0,0034 на последней, то есть признаков переобучения нет. Наилучшей по функции потерь на проверочной выборке оказывается последняя, 84-я эпоха, поэтому ранняя остановка в итоговом запуске не срабатывает. Доля правильных ответов на проверочной выборке достигает единицы на 42-й эпохе, после чего продолжает убывать только функция потерь.

4.2 Итоговое качество

Результат прогноза на проверочной выборке из 113 примеров при зерне 42 приведён в таблице 3.

Таблица 3 — Качество итоговой модели и матрица ошибок на проверочной выборке

Бинарная кросс-энтропия	0,0111	
Доля правильных ответов	1,0000	
	Прогноз B	Прогноз M
Фактически B	71	0
Фактически M	0	42

ЗАКЛЮЧЕНИЕ

В ходе работы разработан многослойный персептрон с настраиваемой архитектурой и обучен решению задачи бинарной классификации диагноза. Реализованы прямое распространение сигнала, бинарная кросс-энтропия и две другие функции потерь, обратное распространение ошибки и градиентный спуск с возможностью накопления импульса.

Обратный проход построен на обратном автоматическом дифференцировании: граф вычислений формируется как побочный продукт прямого прохода, а производные по параметрам выводятся программно из правил для элементарных операций.

Архитектура выбрана экспериментально сравнением шести конфигураций, усреднённых по пяти запускам. Установлено, что в рассматриваемой задаче ёмкость сети не является ограничивающим фактором: изменение числа параметров в двенадцать раз не приводит к значимому изменению результата, а добавление третьего скрытого слоя его ухудшает. Определяющим параметром оказалась функция активации скрытых слоёв: замена сигмоиды на гиперболический тангенс снижает функцию потерь на проверочной выборке втрое при неизменном числе параметров, что объясняется четырёхкратной разницей в максимуме производной и симметричностью тангенса относительно нуля. Функция `ReLU` при этом даёт наилучшее приближение обучающей выборки, но заметно переобучается и показывает втрое больший разброс по сиду генератора случайных чисел.

В качестве итоговой принята конфигурация из двух скрытых слоёв по 24 нейрона с гиперболическим тангенсом, содержащая 1394 параметра. На проверочной выборке она даёт бинарную кросс-энтропию 0,0111; усреднение по пяти сидам даёт 0,0265 и 0,9858 соответственно, при постоянном ответе 0,6283. Признаков переобучения не наблюдается: разрыв между функциями потерь на обучающей и проверочной выборках сокращается на протяжении всего обучения.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинги ключевых фрагментов реализации

```
struct tensor_node;

using node_pointer = std::shared_ptr<tensor_node>;

struct tensor_node
{
    matrix value;
    matrix grad;
    bool requires_grad{false};
    std::string op{"leaf"};
    std::vector<node_pointer> inputs;
    std::function<void(const matrix &)> propagate;
};
```

```
[[nodiscard]] inline tensor matmul(const tensor& left, const tensor& right)
{
    detail::require_defined(left, "matmul");
    detail::require_defined(right, "matmul");

    if (left.cols() != right.rows())
        throw std::invalid_argument(fmt::format("matmul: cannot multiply {} by {}",
                                                shape_string(left.value()),
                                                shape_string(right.value())));

    return detail::make_node(
        left.value() * right.value(), "matmul", {left, right},
        [](const matrix& outer, tensor_node&, const std::vector<tensor_node*>& inputs)
        {
            if (inputs[0]->requires_grad)
                inputs[0]->grad += outer * inputs[1]->value.transpose();
            if (inputs[1]->requires_grad)
                inputs[1]->grad += inputs[0]->value.transpose() * outer;
        });
}
```

```
[[nodiscard]] inline tensor softmax_rows(const tensor& operand)
{
    detail::require_defined(operand, "softmax_rows");

    const matrix& input = operand.value();
    if (input.cols() == 0)
        throw std::invalid_argument("softmax_rows: the input has no columns");

    const matrix exponentials = (input.colwise() - input.rowwise().maxCoeff()).array().exp();
    const matrix totals = exponentials.rowwise().sum();

    return detail::make_node(
        exponentials.array().colwise() / totals.col(0).array(), "softmax", {operand},
        [](const matrix& outer, tensor_node& self, const std::vector<tensor_node*>& inputs)
        {
```

```
            if (!inputs[0]->requires_grad)
                return;

            const matrix weighted = outer.cwiseProduct(self.value);
            inputs[0]->grad += (weighted.array() - self.value.array().colwise() *
                                                   weighted.rowwise().sum().col(0).array())
                                   .matrix();
        });
}
```

```
void backward(const matrix &seed) const
{
    require_defined("backward");
    if (seed.rows() != m_node->value.rows() || seed.cols() != m_node->value.cols())
        throw std::invalid_argument(
            fmt::format("tensor::backward: seed {} does not match the root {}",
                        shape_string(seed), shape_string(m_node->value)));

    const std::vector<tensor_node *> order = reverse_topological_order(m_node.get());

    for (tensor_node *each : order)
        if (!each->inputs.empty())
            each->grad.fill(0.0);

    m_node->grad += seed;

    for (tensor_node *each : order)
        if (each->propagate)
            each->propagate(each->grad);
}
```

```
void step(const std::vector<tensor>& parameters) override
{
    const bool use_momentum = m_momentum > 0.0;
    if (use_momentum && m_velocities.size() != parameters.size())
    {
        m_velocities.clear();
        m_velocities.reserve(parameters.size());
        for (const tensor& parameter : parameters)
            m_velocities.push_back(
                matrix::Zero(parameter.value().rows(), parameter.value().cols()));
    }

    for (std::size_t i = 0; i < parameters.size(); ++i)
    {
        matrix& value = parameters[i].mutable_value();
        const matrix& gradient = parameters[i].grad();

        if (value.rows() != gradient.rows() || value.cols() != gradient.cols())
            throw std::invalid_argument(
                fmt::format("sgd::step: parameter {} is {} but its gradient is {}", i,
                            shape_string(value), shape_string(gradient)));

        if (use_momentum)
        {
            matrix& velocity = m_velocities[i];
            velocity = m_momentum * velocity - m_learning_rate * gradient;
            value += velocity;
        }
```

```
        else
        {
            value -= m_learning_rate * gradient;
        }
    }
}
```

```
for (std::size_t epoch = 0; epoch < m_config.epochs; ++epoch)
{
    const std::vector<std::size_t> order = rng.permutation(sample_count);

    for (std::size_t start = 0; start < sample_count; start += m_config.batch_size)
    {
        const std::size_t stop = std::min(start + m_config.batch_size, sample_count);
        const std::vector<std::size_t> batch(
            order.begin() + static_cast<std::ptrdiff_t>(start),
            order.begin() + static_cast<std::ptrdiff_t>(stop));

        zero_grad(parameters);

        const tensor predictions =
            network.forward(tensor::constant(train_inputs(batch, Eigen::all)));
        const tensor batch_loss = (*loss)(predictions, train_targets(batch, Eigen::all));

        batch_loss.backward();
        optimizer.step(parameters);
    }

}
```